

Azure Function that sends an email to Gmail using SMTP (Gmail) with .NET (C#).

cover:

1. Prerequisites
 2. Gmail configuration (important)
 3. Create Azure Function (HTTP trigger)
 4. Write email-sending code
 5. Local testing
 6. Deploy to Azure
 7. Invoke the function
-

1. Prerequisites

Ensure you have:

- Visual Studio 2022 (with **Azure development** workload)
 - .NET 8 (or .NET 6 LTS)
 - Azure account
 - Gmail account
-

2. Gmail Configuration (Very Important)

Google **does not allow normal passwords** for SMTP anymore.

Step 1: Enable 2-Step Verification

- Go to **Google Account** → **Security**
- Enable **2-Step Verification**

Step 2: Create App Password

- Security → App passwords
- App: **Mail**
- Device: **Windows Computer**

- Copy the generated **16-digit password**

You will use this password in Azure Function.

3. Create Azure Function App (Visual Studio)

Step 1: Create Project

- Open **Visual Studio**
- **Create New Project**
- Select **Azure Functions**
- Click **Next**

Step 2: Project Settings

- Project name: **EmailFunctionApp**
- Runtime: **.NET 8**
- Trigger: **HTTP trigger**
- Authorization level: **Function**

Click **Create**

4. Add Required NuGet Package

Azure Functions already includes most dependencies.

If not present, add:

```
dotnet add package System.Net.Mail
```

(No third-party library required.)

5. Write Azure Function Code (Email Sender)

SendEmailFunction.cs

```
using System.Net;
```

```
using System.Net.Mail;
```

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.Azure.WebJobs;
using Microsoft.Azure.WebJobs.Extensions.Http;
using Microsoft.AspNetCore.Http;
using Microsoft.Extensions.Logging;
using System.Threading.Tasks;

namespace EmailFunctionApp
{
    public static class SendEmailFunction
    {
        [FunctionName("SendEmail")]
        public static async Task<ActionResult> Run(
            [HttpTrigger(AuthorizationLevel.Function, "post", Run = true)] HttpRequest req,
            ILogger log)
        {
            log.LogInformation("SendEmail function triggered.");

            // Read query parameters
            string toEmail = req.Query["to"];
            string subject = req.Query["subject"];
            string body = req.Query["body"];

            if (string.IsNullOrEmpty(toEmail))
            {
                return new BadRequestObjectResult("Recipient email is required.");
            }
        }
    }
}
```

```
}
```

```
try
```

```
{
```

```
    var smtpClient = new SmtpClient("smtp.gmail.com", 587)
```

```
    {
```

```
        Credentials = new NetworkCredential(
```

```
            "yourgmail@gmail.com",
```

```
            "YOUR_GMAIL_APP_PASSWORD"
```

```
        ),
```

```
        EnableSsl = true
```

```
    };
```

```
    var mailMessage = new MailMessage
```

```
    {
```

```
        From = new MailAddress("yourgmail@gmail.com"),
```

```
        Subject = subject ?? "Azure Function Email",
```

```
        Body = body ?? "Hello from Azure Function",
```

```
        IsBodyHtml = false
```

```
    };
```

```
    mailMessage.To.Add(toEmail);
```

```
    await smtpClient.SendMailAsync(mailMessage);
```

```
    return new OkObjectResult("Email sent successfully.");
```

```
    }  
    catch (System.Exception ex)  
    {  
        log.LogError(ex, "Error sending email");  
        return new StatusCodeResult(500);  
    }  
}  
}  
}
```

6. Secure Credentials (Best Practice)

 Do NOT hardcode credentials in real projects

 Use `local.settings.json`

`local.settings.json`

```
{  
  "IsEncrypted": false,  
  "Values": {  
    "AzureWebJobsStorage": "UseDevelopmentStorage=true",  
    "FUNCTIONS_WORKER_RUNTIME": "dotnet",  
    "GmailUser": "yourgmail@gmail.com",  
    "GmailPassword": "YOUR_GMAIL_APP_PASSWORD"  
  }  
}
```

Update Code to Read from Configuration

```
string gmailUser = Environment.GetEnvironmentVariable("GmailUser");  
string gmailPassword = Environment.GetEnvironmentVariable("GmailPassword");
```

```
var smtpClient = new SmtpClient("smtp.gmail.com", 587)
{
    Credentials = new NetworkCredential(gmailUser, gmailPassword),
    EnableSsl = true
};
```

7. Run & Test Locally

Start the Function

- Press **F5**
- Copy the generated URL, example:

http://localhost:7071/api/SendEmail

Test Using Browser / Postman

POST http://localhost:7071/api/SendEmail?to=test@gmail.com&subject=Hello&body=TestMail

 Email should arrive in Gmail inbox.

8. Deploy to Azure

Step 1: Publish

- Right-click project → **Publish**
- Target: **Azure**
- Select **Azure Function App**
- Create new Function App

Step 2: Configure App Settings (Azure Portal)

- Go to **Function App** → **Configuration**
- Add:
 - GmailUser

- GmailPassword

Restart the Function App.

9. Invoke Azure Function (Cloud)

`https://<your-function-name>.azurewebsites.net/api/SendEmail?to=test@gmail.com&subject=Azure&body=Hello`

Include function key if authorization level is **Function**.

10. Summary Architecture

Client (Postman / API / Browser)

|

v

Azure Function (HTTP Trigger)

|

v

Gmail SMTP (smtp.gmail.com)

|

v

Recipient Inbox
